76. Minimum Window Substring Hard

Acceptance: 46.7% | LeetCode

Given two strings `s` and `t` of lengths `m` and `n` respectively, return *the**minimum window substring** of `s` such that every character in `t` (**including duplicates**) is included in the window*. If there is no such substring, return *the empty string* `""`.

The testcases will be generated such that the answer is **unique**.

Example 1:

```
**Input:** s = "ADOBECODEBANC", t = "ABC"
**Output:** "BANC"
**Explanation:** The minimum window substring "BANC"
includes 'A', 'B', and 'C' from string t.
```

Example 2:

```
**Input:** s = "a", t = "a"
**Output:** "a"
**Explanation:** The entire string s is the minimum
window.
```

Example 3:

```
**Input:** s = "a", t = "aa"
**Output:** ""
**Explanation:** Both 'a's from t must be included in
the window.
Since the largest window of s only has one 'a', return
empty string.
```

Constraints:

- `m == s.length`
- `n == t.length`
- `1 <= m, n <= 105`
- `s` and `t` consist of uppercase and lowercase English letters.

Follow up: Could you find an algorithm that runs in `O(m + n)` time?

Claude Opus Solution

Minimum Window Substring

1. Intuition

The key insight is the **sliding window** pattern with two pointers:

- We need a contiguous substring of `s` that contains **all** characters of `t` (including duplicates).
- A brute force approach checking every substring would be O(m²·n) — too slow.
- The clever realization: **expand** the window to the right until it's valid (contains all of `t`), then **contract** from the left to make it as small as possible while still valid.

The crucial trick is using a counter that tracks **how many characters we still need**. By allowing counts to go negative, we naturally track "surplus" characters that can be safely removed during contraction.

---

2. Approach (Pseudocode)

```
1. Build a frequency map `need` of all characters in t.
2. Track `missing` = number of characters still required (starts at len(t)).
3. Use two pointers: left (current) and right (index).
4. For each character as we move right:
     a. If this char was needed (count > 0), decrement missing.
     b. Decrement its count in the map (can go negative = surplus).
5. When missing == 0, we have a valid window:
     a. Contract left while the leftmost char is in surplus (count < 0).
     b. Record the window if it's the smallest so far.
     c. Move left forward by one (this breaks validity, so missing++).
6. Return the best recorded window.
```

---

3. Python Solution

```python
import collections


class Solution:
    def minWindow(self, s: str, t: str) -> str:
        # Edge case: t longer than s can never have a valid window
```

```
        if len(t) > len(s):
            return ""

        # `need[c]` = how many of char c we still need in the window.
        # Negative value means we have a SURPLUS of that character.
        need = collections.Counter(t)

        # `missing` = total count of characters still required.
        # When it hits 0, the current window is valid.
        missing = len(t)

        # Result window boundaries; (0,0) means "no window found yet".
        best_start, best_end = 0, 0

        # `left` is the left edge of our sliding window.
        left = 0

        # `right` (via enumerate starting at 1) is the right edge (exclusive).
        # Using 1-based index so the window is s[left:right].
        for right, char in enumerate(s, 1):
            # --- EXPAND: include s[right-1] into the window ---
            # If we still needed this char, we've covered one requirement.
            if need[char] > 0:
                missing -= 1
            # Decrement count: a negative value tracks surplus characters.
            need[char] -= 1

            # --- VALID WINDOW FOUND ---
            if missing == 0:
                # --- CONTRACT: shrink from the left as much as possible ---
                # We can drop a left char if it's in surplus (need < 0).
                while left < right and need[s[left]] < 0:
                    need[s[left]] += 1  # giving back a surplus char
                    left += 1

                # Record this window if it's the first or smaller than best.
                if best_end == 0 or (right - left) < (best_end - best_start):
                    best_start, best_end = left, right

                # --- MOVE ON: remove the critical left char to seek a new window ---
                # s[left] is now a REQUIRED char (count was 0), so removing it
                # breaks validity: missing goes back up by 1.
                need[s[left]] += 1
                missing += 1
                left += 1

        # If best_end is still 0, no valid window was ever found.
        return s[best_start:best_end]
```

---

4. Complexity Analysis

Time Complexity: O(m + n) - Building the counter from `t` takes O(n). - The `right` pointer traverses `s` once → O(m). - The `left` pointer also moves forward at most m times total across all iterations (it never resets backward), so the inner `while` loop is amortized O(m). - Total: **O(m + n)**.

Space Complexity: O(1) (or O(k)) - The `need` counter holds at most 52 distinct characters (uppercase + lowercase English letters), so it's bounded by a constant. - Technically O(k) where k = distinct chars, but since k ≤ 52, it's effectively **O(1)**.

---

5. Alternative Approaches

A. Classic Two-Pointer with "formed" tracking

A more textbook sliding-window variant uses two counters and a `formed` variable counting how many *unique* required characters are fully satisfied:

```
def minWindow(self, s: str, t: str) -> str:
    if not s or not t:
        return ""
    need = collections.Counter(t)
    required = len(need)        # unique chars needed
```

```
    formed = 0                  # unique chars currently satisfied
    window = collections.defaultdict(int)
    left = 0
    best = (float('inf'), 0, 0) # (length, left, right)

    for right, c in enumerate(s):
        window[c] += 1
        # This char's requirement is now fully met
        if c in need and window[c] == need[c]:
            formed += 1
        # Try shrinking while all requirements are satisfied
        while left <= right and formed == required:
            if right - left + 1 < best[0]:
                best = (right - left + 1, left, right)
            window[s[left]] -= 1
            if s[left] in need and window[s[left]] < need[s[left]]:
                formed -= 1
            left += 1
    return "" if best[0] == float('inf') else s[best[1]:best[2]+1]
```

This is more intuitive to read but slightly more verbose. Same O(m+n) time.

B. Optimization with filtered string

When `t` is small relative to `s`, you can pre-filter `s` to only the indices containing characters in `t`, then slide over that shorter list. This helps when most of `s` is irrelevant, but doesn't change worst-case complexity.

Key Pattern Takeaway: The "negative count = surplus" trick in the reference solution elegantly merges the validity check and contraction logic into one counter, which is why it's so compact. The mental model is: *expand to satisfy, contract to minimize, then deliberately break to search for the next candidate.*

Codex Solution

1. Intuition

This is a classic **sliding window** problem.

We need the smallest substring of `s` that contains all characters of `t`, including duplicates.

For example:

```
s = "ADOBECODEBANC"
t = "ABC"
```

A valid window must contain:

```
A: 1
B: 1
C: 1
```

The key idea:

1. Use a right pointer to **expand** the window until it contains all required characters.
2. Once the window is valid, use a left pointer to **shrink** it as much as possible.
3. Keep track of the smallest valid window seen.

This works because each pointer only moves forward, so the algorithm is linear.

---

2. Approach

Pseudocode

```
count required characters in t using need map

window = empty character count map
required = number of unique characters in t
formed = 0

left = 0
best_length = infinity
```

```
best_start = 0

for right from 0 to len(s) - 1:
    add s[right] to window

    if s[right] is needed and window count matches need count:
        formed += 1

    while formed == required:
        update best answer if current window is smaller

        remove s[left] from window

        if s[left] is needed and window count becomes less than need count:
            formed -= 1

        left += 1

if no valid window was found:
    return ""

return best substring
```

---

3. Python Solution

```python
from collections import Counter, defaultdict


class Solution:
    def minWindow(self, s: str, t: str) -> str:
        # If t is longer than s, it is impossible for s to contain t.
        if len(t) > len(s):
            return ""

        # need stores how many times each character must appear.
        #
        # Example:
        # t = "AABC"
        # need = {
        #   'A': 2,
        #   'B': 1,
        #   'C': 1
        # }
        need = Counter(t)

        # window stores character counts in the current sliding window.
        window = defaultdict(int)

        # required is the number of unique characters that must be satisfied.
        #
        # For t = "AABC", required = 3 because we need A, B, and C.
        required = len(need)

        # formed counts how many unique required characters currently have
        # enough frequency inside the window.
        #
        # A character is considered "formed" only when:
        # window[char] == need[char]
        formed = 0

        # Left pointer of the sliding window.
        left = 0

        # Best answer tracking.
        best_len = float("inf")
        best_start = 0

        # Expand the window by moving right.
        for right, char in enumerate(s):
            # Add current character to the window.
            window[char] += 1
```

```
            # If this character is required and its count now exactly matches
            # the needed count, then this character requirement is satisfied.
            if char in need and window[char] == need[char]:
                formed += 1

            # If all required characters are satisfied, try shrinking.
            while formed == required:
                current_len = right - left + 1

                # Update best answer if this window is smaller.
                if current_len < best_len:
                    best_len = current_len
                    best_start = left

                # Remove the leftmost character from the window.
                left_char = s[left]
                window[left_char] -= 1

                # If removing this character makes the window invalid,
                # decrease formed.
                if left_char in need and window[left_char] < need[left_char]:
                    formed -= 1

                # Move left pointer forward to shrink the window.
                left += 1

        # If best_len was never updated, no valid window exists.
        if best_len == float("inf"):
            return ""

        return s[best_start:best_start + best_len]
```

---

4. Complexity Analysis

Let:

```
m = len(s)
n = len(t)
```

Time Complexity: `O(m + n)`

Explanation:

- Building the `Counter` for `t` takes `O(n)`.
- The right pointer moves through `s` once: `O(m)`.
- The left pointer also moves through `s` at most once: `O(m)`.

So the total time complexity is:

```
O(n + m + m) = O(m + n)
```

Space Complexity: `O(1)`

Since `s` and `t` contain only uppercase and lowercase English letters, there are at most 52 possible characters.

Therefore, the hash maps store at most 52 keys.

So the space complexity is technically:

```
O(1)
```

If the character set were unlimited, the space complexity would be:

```
O(k)
```

where `k` is the number of unique characters in `t`.

---

5. Alternative Approaches

1. Brute Force

Check every substring of `s` and test whether it contains all characters of `t`.

This would be too slow.

Time complexity:

```
O(m^2 * alphabet_size)
```

or worse depending on implementation.

---

2. Sliding Window with a Missing Counter

Instead of tracking `formed` and `required`, we can track a single variable `missing`, meaning how many total characters from `t` are still missing.

This is slightly more compact, but the logic can be less intuitive.

---

3. Filtered Sliding Window

We can first filter `s` to keep only characters that appear in `t`.

Example:

```
s = "ADOBECODEBANC"
t = "ABC"

filtered = [(0, 'A'), (3, 'B'), (5, 'C'), (9, 'B'), (10, 'A'), (12, 'C')]
```

Then run sliding window only on this filtered list.

This can be faster in practice when `s` is very large and most characters are irrelevant.

Worst-case complexity is still:

```
O(m + n)
```